MimeSys Algorithms

Package version 0.99.0

27 October 2026

Contents

1. Notation . . . 1
2. Input and preparation . . . 2
   - 2.1 Containers . . . 2
   - 2.2 Sample matching . . . 2
   - 2.3 Missing values and constant features . . . 3
   - 2.4 View transforms . . . 3
3. Fitting the template . . . 3
   - 3.1 Marginals: empirical quantile functions . . . 3
   - 3.2 Normal scores . . . 4
   - 3.3 Within-view factor model . . . 4
   - 3.4 Shared cross-view space (MAXVAR generalized CCA) . . . 4
   - 3.5 Choosing the shared rank: sequential parallel analysis . . . 5
   - 3.6 Two-part summaries and composition type . . . 6
   - 3.7 What the template stores . . . 6
4. Null multi-omics feature generation . . . 6
   - 4.1 The generative model . . . 6
   - 4.2 Properties: why crosstalk is a clean dial . . . 7
   - 4.3 Feature counts . . . 7
   - 4.4 Generating the data . . . 8
   - 4.5 Two-part model for sparse microbiome views . . . 8
   - 4.6 Composition constraint . . . 8
   - 4.7 Back-transformation . . . 8
   - 4.8 Output . . . 9
5. Outcome generation (`gen_simmba()`) . . . 9
   - 5.1 Standardized features . . . 9
   - 5.2 Signal coefficients . . . 9
   - 5.3 Linear predictor and signal balance . . . 10
   - 5.4 Outcome models (`outcome.model`) . . . 10
   - 5.5 How the signal controls interact . . . 11
   - 5.6 Outcome types . . . 12
   - 5.7 Truth and split . . . 13
6. Crosstalk calibration and template validation . . . 13
   - 6.1 Block distance . . . 13

6.2 Calibration (`calibrate_crosstalk()`) . . . 14
6.3 Validation (`validate_template()`) . . . 14
7. Mediation simulation (`gen_mediation()`) . . . 14
7.1 Exposure and mediators . . . 14
7.2 Exposure to mediators (a-paths) . . . 15
7.3 Outcome (b-paths) . . . 15
7.4 Natural effects . . . 15
7.5 Path sizes and exact calibration . . . 16
7.6 Paired counterfactual Monte Carlo . . . 16
7.7 Binary outcome . . . 17
7.8 Observed data and truth . . . 17
8. Sequential mediation . . . 17
8.1 Latent model . . . 17
8.2 Links . . . 18
8.3 Natural path-specific effects . . . 18
8.4 Scenarios . . . 19
8.5 Estimation on the observed scale . . . 19
9. Multi-study simulation (`gen_multistudy()`) . . . 20
9.1 Features of every study . . . 20
9.2 Study coefficients . . . 20
9.3 Predictor and outcome . . . 21
9.4 Random streams . . . 21
9.5 Validation . . . 21
10. MAE wrappers and reproducibility . . . 21
11. Worked example: the Stelzer template . . . 22
12. Default settings . . . 22
13. Assumptions and limitations . . . 23
References . . . 23

This document describes, with the mathematics, every algorithm in MimeSys (version 0.99.0): how the template is fitted, how null multi-omics data are generated, how outcome signal is planted, how crosstalk is calibrated and the template validated, how several heterogeneous studies are simulated, and how parallel and sequential mediation are simulated with exact causal truth. Each section states what the code does, the model behind it, and why it is built that way.

The main separation is:

- **Null feature generation** creates realistic multi-view feature matrices: each feature's distribution, the dependence within each view, and the dependence between views.
- **Outcome generation** plants an outcome with known drivers *after* the features exist (`gen_simmba()`).
- **Mediation generation** lets an exposure act on chosen features *before* they are mapped to the data scale, and then generates an outcome from them (`gen_mediation()`).
- **Multi-study generation** simulates several studies on the same features, from one template or one template per cohort, with controlled case-mix, batch, coefficient, intercept and SNR differences (`gen_multistudy()`).

Crosstalk controls only cross-view feature dependence; the outcome model controls what predicts the outcome. High crosstalk does not by itself make integration methods better: that depends on how the signal is distributed across views.

1. Notation

Symbol	Meaning
M	number of views, indexed by m
n_0, n	number of template samples (after matching), number of simulated samples
p_{0m}, p_m	template and simulated feature counts of view m
$X_m \in \mathbb{R}^{n_0 \times p_{0m}}$	template matrix of view m on the *fitted* scale (after transform, imputation and filtering)
Q_{mj}	empirical quantile function of feature j of view m
Z_m	normal scores of X_m
k_m	within-view rank (number of factors)
$\Lambda_m \in \mathbb{R}^{p_{0m} \times k_m}$	within-view loadings; Λ_{mj} is the row of feature j; $h_{mj} = \|\Lambda_{mj}\|^2$ its communality
$F_m \in \mathbb{R}^{n_0 \times k_m}$	template factor scores, $F_m^\top F_m/(n_0-1) = I$
r	shared rank (number of shared cross-view variates)
$S \in \mathbb{R}^{n_0 \times r}$	template shared variates, $S^\top S/(n_0-1) = I$
$B_m \in \mathbb{R}^{k_m \times r}$	cross-view loadings, $B_m = F_m^\top S/(n_0-1)$
γ_m	crosstalk weight of view m
$s \sim N(0, I_r)$, f_m, z_{mj}	simulated shared variates, view factors and latent feature scores
Φ, Φ^{-1}	standard normal CDF and quantile function
$\mathrm{std}(v)$	v centred and scaled to standard deviation 1

2. Input and preparation

2.1 Containers

The input is a `MultiAssayExperiment` (MAE), a single `SummarizedExperiment`, or a named list of matrices. Internally every view is a numeric matrix with **samples in rows and features in columns**; MAE and SummarizedExperiment assays (features in rows) are transposed, including MAE experiments stored as plain matrices. Missing row names become `sample_1, sample_2, ...`, missing feature names `<view>_feature_1, ...`, and duplicated feature names are made unique. `validate_mimesys_input()` reports samples, features, missing values and constant features per view before fitting.

2.2 Sample matching

For an MAE, each experiment's column names are replaced by the primary sample IDs of its `sampleMap()`, so experiments that name the same subject differently are still matched (several columns for one primary sample are an error). The samples used are the intersection of the sample IDs of all views:

$$\mathcal{J} = \bigcap_{m=1}^{M} \mathrm{rownames}(X_m), \qquad n_0 = |\mathcal{J}| \geq 10.$$

Duplicate IDs within a view are an error. A list of matrices without row names is matched by position (all views receive `sample_1, ...`), which requires every view to have the same number of rows; views without sample IDs and with different numbers of rows are an error. Give matrices explicit sample IDs whenever the order is not guaranteed to match.

The template assumes **one sample per subject**. Repeated measures are not independent and inflate the estimated cross-view structure: all 150 samples of the full Stelzer data (3 timepoints for 53 subjects) give 18 shared factors, one timepoint per subject gives 2.

2.3 Missing values and constant features

Per view: features missing in every sample are dropped (with a message); features more than 50% missing give a warning; remaining missing values are replaced by the feature median, $x_{ij} \leftarrow \text{median}_i(x_{ij})$; constant features are dropped. A view with no non-constant features is an error.

2.4 View transforms

`view.transform` is `"none"`, `"log1p"` or `"clr"` per view (unnamed views use `"none"`; unknown view names are errors).

log1p. With $x_{\min}$ the smallest value of the view,

$$\tilde{x}_{ij} = \log(1 + x_{ij} + c), \qquad c = \begin{cases} |x_{\min}| + 1 & x_{\min} < -1, \\ 0 & \text{otherwise.} \end{cases}$$

Centred log-ratio. The view must be non-negative. With pseudocount ϵ equal to half the smallest positive value of the view,

$$\tilde{x}_{ij} = \log(x_{ij} + \epsilon) - \frac{1}{p} \sum_{l=1}^{p} \log(x_{il} + \epsilon).$$

The template is fitted, calibrated and validated on the transformed ("fitted") scale. For transformed views it also keeps each feature's original-scale sorted values so that simulations can be returned on the measurement scale (Section 4.7).

3. Fitting the template

`fit_template()` learns the three ingredients of the generative model of Section 4: the marginals Q_{mj}, the within-view loadings Λ_m, and the cross-view loadings B_m.

3.1 Marginals: empirical quantile functions

For feature j the sorted template values $x_{(1)} \le ... \le x_{(n_0)}$ are stored. A uniform $u \in (0, 1)$ is mapped to data by

$$Q(u) = \begin{cases} x_{(\lceil u\, n_0 \rceil)} & \text{integer-valued feature,} \\ x_{(\lfloor h \rfloor)} + (h - \lfloor h \rfloor)(x_{(\lfloor h \rfloor + 1)} - x_{(\lfloor h \rfloor)}) & \text{otherwise,} \end{cases} \qquad h = (n_0 - 1)u + 1,$$

a step function for integer-valued features (so counts stay integers) and linear interpolation otherwise.

A feature is integer-valued when all its values are within 10^{-8} of an integer. Zeros, point masses, bounds, skewness and heavy tails therefore carry over exactly, and simulated values never leave the observed range.

3.2 Normal scores

Each feature is converted to normal scores by a randomized probability integral transform,

$$z_{ij} = \Phi^{-1}\left(\frac{\mathrm{rank}(x_{ij})}{n_0 + 1}\right),$$

with ties broken at random, and each column is then centred and scaled to unit variance. Random tie-breaking spreads a point mass (for example many zeros) over a range of normal scores, so tied features still have well-defined latent correlations; it is also why template fitting uses a seed.

3.3 Within-view factor model

For view m, with the singular value decomposition $Z_m = UDV^\top$ and its leading k_m triplets (U_k, D_k, V_k),

$$\Lambda_m = \frac{V_k D_k}{\sqrt{n_0 - 1}}, \qquad F_m = U_k\sqrt{n_0 - 1}, \qquad Z_m \approx F_m \Lambda_m^\top.$$

The scores have identity sample covariance and Λ_{mj} is the vector of correlations between feature j and the factors.

Rank. With d_l the singular values,

$$k^* = \begin{cases} \min\left\{k : \sum_{l\le k} d_l^2 \big/ \sum_l d_l^2 \;\ge\; v\right\} & \texttt{rank = NULL}, \\ \lceil 2\log p_{0m} \rceil & \texttt{rank = "2logp"}, \\ \texttt{rank}_m & \text{given}, \end{cases}$$

with $v =$ `var.explained` (default 0.7),

and $k_m = \max\{1, \min(k^*, \texttt{max.rank}, p_{0m} - 1)\}$ with `max.rank` $= \max(1, \lfloor n_0/3 \rfloor)$ by default; the cap also applies to an explicit rank.

Communality cap. If $h_{mj} = \|\Lambda_{mj}\|^2 > 0.99$, the row is rescaled, $\Lambda_{mj} \leftarrow \Lambda_{mj}\sqrt{0.99/h_{mj}}$, so every feature keeps at least 1% unique variance.

3.4 Shared cross-view space (MAXVAR generalized CCA)

Stack the view scores, $F = [F_1, \ldots, F_M]$, and take the eigendecomposition of their correlation matrix,

$$C = \mathrm{Cor}(F) = W\,\Omega\,W^\top, \qquad \omega_1 \ge \omega_2 \ge \ldots$$

Because each F_m is orthonormal, the diagonal blocks of C are identities and the off-diagonal blocks are the cross-correlation matrices between views' factor scores.

What the eigenvalues mean. Let $P_m = F_m F_m^\top/(n_0 - 1)$, the projection onto view m's factor space. The non-zero eigenvalues of C are those of $\sum_m P_m$, and for a standardized variate s in the stacked score space

$$\frac{s^\top(\sum_m P_m)s}{s^\top s} = \sum_{m=1}^{M} R^2(s \mid F_m),$$

the sum over views of the squared multiple correlation of s with each view's factors. The leading eigenvector is therefore the variate best explained by all views together (the MAXVAR criterion), and its eigenvalue is that summed R^2: at most M, and about 1 for a direction that lives in one view only. For two views the eigenvalues are $1 \pm \rho_i$, with ρ_i the canonical correlations between the two sets of factor scores.

Shared variates and loadings. For a shared rank $r > 0$, the shared variates are the standardized projections $S = \mathrm{std}(FW_{1:r})$, orthonormalized by a QR decomposition and scaled so that $S^\top S/(n_0 - 1) = I_r$. The cross-view loadings are

$$B_m = \frac{F_m^\top S}{n_0 - 1} \qquad (k_m \times r),$$

the regression coefficients of the shared variates on view m's factors. Column l has squared norm $R^2(S_l \mid F_m) \le 1$; more generally $B_m^\top B_m = S^\top P_m S/(n_0 - 1) \preceq I$, so $\sigma_{\max}(B_m) \le 1$. The per-sample scores F_m are discarded after fitting.

3.5 Choosing the shared rank: sequential parallel analysis

With `shared.rank = NULL`, r is chosen by a sequential (deflated) permutation test. Start with $F_m^{(0)} = F_m$ and $r = 0$:

1. Compute the largest eigenvalue λ_{obs} of $\mathrm{Cor}([F_1^{(r)}, \dots, F_M^{(r)}])$.
2. For $b = 1, \dots, B$ (`nperm`, default 200), permute the rows of each view's score matrix independently (this destroys cross-view dependence but keeps within-view structure) and record the largest eigenvalue $\lambda^{(b)}$.
3. Compute the step p-value and the 95% null threshold:

$$p_{r+1} = \frac{1 + \#\{b : \lambda^{(b)} \ge \lambda_{\mathrm{obs}}\}}{B + 1}, \qquad t_{r+1} = \mathrm{quantile}_{0.95}(\lambda^{(1)}, \dots, \lambda^{(B)}).$$

4. If $p_{r+1} \ge 0.05$, stop: the shared rank is r.
5. Otherwise set $r \leftarrow r + 1$ and **deflate**: with v the leading eigenvector, form $\hat{s} = \mathrm{std}([F_1^{(r-1)}, \dots, F_M^{(r-1)}]\, v)$, remove it from every view,

$$F_m \leftarrow F_m - \hat{s}\, \frac{\hat{s}^\top F_m}{\hat{s}^\top \hat{s}},$$

and re-orthonormalize each view by keeping all but one of its left singular vectors (scaled by $\sqrt{n_0 - 1}$). Go back to step 1.

The loop also stops when r reaches $\min_m k_m$. The p-values, thresholds and tested eigenvalues are stored as `gcca.pvalue`, `gcca.threshold` and `gcca.step.eigen`; B_m is then taken from the leading r eigenvectors of the original C.

Why sequential. The classical rule compares the k-th observed eigenvalue with the null of the k-th permuted eigenvalue. Once one real shared factor exists, every later observed eigenvalue is lifted above its permuted counterpart, so that rule over-selects: in simulations with 2, 3 and 5 true shared factors it chose 6 to 15. The correct sequential test compares the *largest remaining* eigenvalue with the null of the *largest* eigenvalue after removing the accepted factors. It recovers the true rank in most runs (with one true factor and $n_0 = 100$ it picks one too many in about 10% of runs, slightly above the nominal 5%, because the deflated direction is estimated).

Borderline warning. A warning is given when the last accepted step has $p > 0.025$ or the first rejected step has $p < 0.08$.

Stability selection (optional). With `shared.rank.runs` $= R > 1$, the whole fit (with fresh tie-breaking and permutations) is repeated and $r = \text{round}(\frac{1}{R}\sum_b r_b)$; the runs are stored in `shared.rank.selection`. An explicit `shared.rank` larger than $\min_m k_m$ is an error.

3.6 Two-part summaries and composition type

For each feature the fit stores its zero probability $\pi_j = \frac{1}{n_0}\sum_i \mathbf{1}(x_{ij} \le 0)$ and the sorted positive values (the conditional quantile function Q_j^+ given presence), used by the two-part model (Section 4.5).

Each view is classified on its original scale as

- **closed**: non-negative with constant row totals, $\text{sd}(\sum_j x_{ij}) \le 10^{-6}\,\text{mean}(\sum_j x_{ij})$; total τ stored (e.g. relative abundances summing to 1);
- **proportion**: all values in $[0, 1]$, row totals at most 1 and more than one feature (a subset of a composition); $\tau = \max_i \sum_j x_{ij}$;
- **none** otherwise.

3.7 What the template stores

Per view: sorted values (fitted and, for transformed views, original scale), integer flags, composition type, two-part summaries, Λ_m, k_m and feature variances. Overall: the fitted matrices (`observed`, used by calibration and validation), B_m, r, the GCCA and parallel-analysis summaries, the transforms and the provenance. Because it keeps the fitted matrices and every feature's values, a template is as sensitive as the source data.

4. Null multi-omics feature generation

`sim_from_template()` generates data with the template's marginals and dependence and no planted signal. Every other simulator uses the same engine.

4.1 The generative model

For each simulated sample, with independent $s \sim N(0, I_r)$, $\epsilon_m \sim N(0, I_{k_m})$ and $e_{mj} \sim N(0, 1)$:

$$
\begin{aligned}
f_m &= \sqrt{\gamma_m}\, B_m s + (I - \gamma_m B_m B_m^\top)^{1/2} \epsilon_m, \\
z_{mj} &= \Lambda_{mj} f_m + \sqrt{\psi_{mj}}\, e_{mj}, \qquad \psi_{mj} = 1 - \|\Lambda_{mj}\|^2, \\
x_{mj} &= Q_{mj}(\Phi(z_{mj})).
\end{aligned}
$$

This is a Gaussian copula with a low-rank factor structure: dependence is defined on the latent normal scale, and each feature is then given its own empirical marginal. The matrix square root is computed from an eigendecomposition with negative eigenvalues set to 0. If $r = 0$ or $\gamma_m = 0$, $f_m = \epsilon_m$.

4.2 Properties: why crosstalk is a clean dial

Proposition. For $\gamma_m \le 1/\sigma_{\max}(B_m)^2$,

$$
\mathrm{Var}(f_m) = I_{k_m}, \qquad \mathrm{Cov}(f_m, f_{m'}) = \sqrt{\gamma_m \gamma_{m'}}\, B_m B_{m'}^\top \ (m \neq m'), \qquad \mathrm{Var}(z_{mj}) = 1.
$$

Proof. $\mathrm{Var}(f_m) = \gamma_m B_m B_m^\top + (I - \gamma_m B_m B_m^\top) = I$; the square root exists exactly when $I - \gamma_m B_m B_m^\top \succeq 0$, i.e. $\gamma_m\, \sigma_{\max}(B_m)^2 \le 1$. The cross-covariance involves only the shared parts because the ϵ's are independent. Finally $\mathrm{Var}(z_{mj}) = \Lambda_{mj}\Lambda_{mj}^\top + \psi_{mj} = 1$. □

Consequences:

1. **Marginals never change.** $\Phi(z_{mj})$ is exactly uniform, so x_{mj} follows the template's empirical quantile function for any crosstalk, sample size or feature count (except for the composition step of Section 4.6).
2. **Within-view structure does not depend on crosstalk.** For features $i \neq j$ of the same view, $\mathrm{Cor}(z_{mi}, z_{mj}) = \Lambda_{mi}\Lambda_{mj}^\top$.
3. **Cross-view latent correlations scale with crosstalk.** For feature i of view m and j of view m',

   $$
   \mathrm{Cor}(z_{mi}, z_{m'j}) = \sqrt{\gamma_m \gamma_{m'}}\, \Lambda_{mi} B_m B_{m'}^\top \Lambda_{m'j}^\top.
   $$

   With a common γ, the whole cross-view correlation matrix is multiplied by γ. At $\gamma = 1$, $B_m B_{m'}^\top = F_m^\top S S^\top F_{m'}/(n_0 - 1)^2$ is the template's cross-covariance of factor scores projected onto the shared space.
4. **Valid range.** `crosstalk_max()` returns $1/\sigma_{\max}(B_m)^2$ per view (always at least 1, because $\sigma_{\max}(B_m) \le 1$); values above 1 amplify the template's shared structure.
5. **Canonical correlations.** The latent canonical correlations between views a and b are the singular values of $\sqrt{\gamma_a \gamma_b}\, B_a B_b^\top$ (`truth$cancor`).
6. **From latent to data correlations.** For continuous marginals the rank (Spearman) correlation of the data is $\frac{6}{\pi}\arcsin(\rho/2)$ for a latent correlation ρ, close to ρ; Pearson correlations of strongly skewed or zero-inflated data are attenuated.

4.3 Feature counts

Fewer features than the template ($p_m \le p_{0m}$): p_m template features are chosen at random (`feature.select = "random"`) or those with the largest fitted-scale variance (`"variance"`), with their rows of Λ_m.

More features ($p_m > p_{0m}$): all template features are kept and $p_m - p_{0m}$ *offspring* features are added. Each offspring has a parent j and a partner l (drawn with replacement), copies the parent's marginal Q_{mj}, and gets loadings

$$\Lambda_{\text{off}} = \sqrt{1-\tau}\,\Lambda_{mj} + \sqrt{\tau}\,\Lambda_{ml}, \qquad \tau = \texttt{feature.jitter} \text{ (default 0.2)},$$

with the communality again capped at 0.99. An offspring resembles its parent without being a copy.

4.4 Generating the data

All views' latent scores $Z_m = F_m \Lambda_m^\top + E_m \operatorname{diag}(\psi_m)^{1/2}$ are built first; an internal hook can modify them jointly before mapping (the mediation simulators use it to let the exposure act and to link features across views). Each column is then mapped through its parent's quantile function, $x_{ij} = Q_{m,\text{parent}(j)}(\Phi(z_{ij}))$.

4.5 Two-part model for sparse microbiome views

With `view.models = c(View = "two_part_copula")`, presence and abundance are generated as two linked parts. For feature j with zero probability π_j:

$$\begin{aligned}
\text{present}_{ij} &= \mathbf{1}(\Phi(z_{ij}) > \pi_j), \\
z^{\text{ab}}_{ij} &= \frac{z_{ij} + 0.15\,\delta_{ij}}{\sqrt{1 + 0.15^2}}, \qquad \delta_{ij} \sim N(0,1), \\
u^*_{ij} &= \frac{\Phi(z^{\text{ab}}_{ij}) - \pi_j}{1 - \pi_j} \quad \text{clipped to } [10^{-8},\, 1 - 10^{-8}], \\
x_{ij} &= \text{present}_{ij} \cdot Q^+_j(u^*_{ij}).
\end{aligned}$$

The share of zeros matches the template exactly. The abundance latent is a lightly perturbed copy of the presence latent (correlation ≈ 0.989), rescaled to unit variance so that its uniform is exact. The rescaling of u^* conditions on presence: a present sample has $\Phi(z) > \pi_j$, so its abundance is drawn from the whole positive distribution (without it, abundances were inflated about three-fold). The model is for sparse non-negative abundances only: views with negative values or fitted on the CLR scale are errors.

4.6 Composition constraint

For compositional views (Section 3.6) simulated samples never exceed the template total τ:

$$x_{i\cdot} \leftarrow \begin{cases} \tau\, x_{i\cdot} \big/ \sum_j x_{ij} & \text{closed, } p_m \ge p_{0m}, \\ x_{i\cdot}\, \min\!\left(1,\ \tau \big/ \sum_j x_{ij}\right) & \text{closed with } p_m < p_{0m}, \text{ or proportion.} \end{cases}$$

In the first case an all-zero row puts the whole total τ on one random feature.

Offspring features in a closed composition therefore dilute the others instead of pushing totals above τ. The constraint is applied on the original scale.

4.7 Back-transformation

With `back.transform = TRUE`, each simulated uniform $u_{ij} = \Phi(z_{ij})$ is mapped through the feature's *original-scale* quantile function instead. Ranks are kept exactly and each feature recovers its original marginal (for clr, including its zeros). For log1p this equals $\exp(\cdot) - 1 - c$ of the fitted value at observed values and interpolates on the original scale between them. Two-part views fitted with log1p use the exact inverse $x = e^{\tilde{x}} - 1 - c$. The default (`FALSE`) returns the fitted scale, which is the scale of calibration, validation and the truth.

4.8 Output

`feature_metadata` has, per feature, its ID and view, the parent template feature, the offspring flag, the observation model, the returned scale, the transform, and the **shared loading** $\ell_j = \Lambda_{mj} B_{m,\cdot 1}$, the latent correlation of the feature with shared variate 1 at $\gamma = 1$. `truth` holds the simulated factor scores, shared variates, crosstalk and canonical correlations.

5. Outcome generation (`gen_simmba()`)

For each replicate, null data are generated (Section 4), then coefficients, a predictor and an outcome; finally the samples are split into training and test sets.

5.1 Standardized features

Within each replicate, over all n samples (before the split),

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\mathrm{sd}(x_j)}$$

(constant columns become 0). Coefficients are therefore effects per standard deviation, and views on very different measurement scales contribute comparably.

5.2 Signal coefficients

In view m with p_m features, the number of signal features is

$$n_m = \max\{1,\ \mathrm{Binomial}(p_m,\ \texttt{signal.prob}_m)\}$$

(0 if $\texttt{signal.prob}_m = 0$). For each signal feature,

$$f_j \sim \mathrm{LogNormal}(\mu, \sigma), \qquad d_j = \begin{cases} -1 & \text{with probability } \texttt{signal.neg.prob}, \\ +1 & \text{otherwise}, \end{cases} \qquad \beta_j = d_j\,|\log_2 f_j|,$$

with $\mu =$ `effect.meanlog` (default 1) and $\sigma =$ `effect.sdlog` (0.4); $\beta_j = 0$ for the other features. With the defaults the typical factor is $e \approx 2.7$ and $|\beta| \approx 1.4$. (This is the log-normal fold-change distribution of the splatter simulator, restricted to the signal features.)

Alignment. With `signal.align` $= \alpha \in [0,1]$, round(αn_m) of the signal features are drawn with probability proportional to ℓ_j^2 and given the sign of ℓ_j; the rest are drawn uniformly among the remaining features. Signals in different views then agree to the extent the views share structure.

5.3 Linear predictor and signal balance

$$\eta_{im} = \sum_{j \in m} \beta_j \tilde{x}_{ij}, \qquad \eta_i^{\text{lin}} = \sum_{m=1}^{M} \eta_{im}.$$

With `signal.share` $= (w_1, \dots, w_M)$, normalized to sum to 1, each view's coefficients are rescaled so that its predictor has variance w_m:

$$\beta_m \leftarrow \beta_m \sqrt{w_m / \mathrm{Var}(\eta_m)}$$

($w_m = 0$ sets the view's coefficients to 0). For example $(1,1,1)$ gives complementary signal of equal size in three views, $(1,0,0)$ puts all linear signal in one view. The correlation matrix of the η_m is returned as `agreement.emp`.

5.4 Outcome models (`outcome.model`)

Mixing two parts. Two standardized components a, b are combined as

$$\eta = \sqrt{1-w}\,\mathrm{std}(a) + \sqrt{w}\,\mathrm{std}(b),$$

so that $\mathrm{Var}(\eta) = 1$ and w is the share of signal variance from b when a and b are uncorrelated.

`outcome.model`	Predictor
`"linear"`	$\eta = \eta^{\text{lin}}$
`"friedman"`	$\eta = F(u_1, \dots, u_5)$
`"friedman2"`	mix of η^{lin} and F with $w =$ `nonlinear.weight` (0.5)
`"shared_latent"`	function of the simulated shared variates (below)
`"interaction"`	mix of η^{lin} and η^{int} with $w =$ `interaction.weight` (0.5)

Friedman. Five signal features $j_1, \dots, j_5$ are chosen (all features if fewer than five have signal; with `friedman.share`, each slot first picks a view with probability equal to its share). They are rank-transformed to the unit interval, $u_{ik} = \mathrm{rank}(x_{ij_k})/(n+1)$, and

$$F(u) = 10\sin(\pi u_1 u_2) + 20(u_3 - 0.5)^2 + 10u_4 + 5u_5,$$

the Friedman (1991) benchmark function.

Shared latent. With $\tilde{s}_k$ the standardized simulated shared variates and $K =$ `shared.latent.n.factors` (default all r),

$$\eta^{\mathrm{SL}} = \mathrm{std}\Big(\frac{1}{\sqrt{K}}\sum_{k=1}^{K}\tilde{s}_{ik}\Big) \quad (\texttt{"linear"}), \qquad \eta = \mathrm{std}\, F(v_{i1},\dots,v_{i5}),\ \ v_{ik} = \frac{\mathrm{rank}(\tilde{s}_{ik})}{n+1} \quad (\texttt{"friedman"}),$$

and `"friedman2"` mixes the two. The outcome then depends on the cross-view latent process rather than on individual features; with crosstalk 0 it is unrelated to the features (a warning is given).

Shared and view-specific latent signal. View m's simulated factor scores are $f_m = \sqrt{\gamma_m}\, B_m s + (I - \gamma_m B_m B_m^\top)^{1/2}\epsilon_m$ (Section 4.1). Their non-shared part

$$p_m = f_m - \sqrt{\gamma_m}\, B_m s = (I - \gamma_m B_m B_m^\top)^{1/2}\epsilon_m$$

is independent of the shared variates s and of the other views. Each view gets one view-specific score, its non-shared factors weighted by loading strength, $q_{im} = \mathrm{std}(\omega_m^\top p_{m,i})$ with $\omega_{mk} \propto \sum_j \lambda_{jk}^2$ (template loadings), and with view weights π_m (`signal.share`, equal by default)

$$\eta_i^{\mathrm{VS}} = \mathrm{std}\Big(\sum_m \sqrt{\pi_m}\, q_{im}\Big), \qquad \eta = \sqrt{w_s}\ \mathrm{std}(\eta^{\mathrm{SL}}) + \sqrt{1 - w_s}\ \eta^{\mathrm{VS}},$$

with w_s = `shared.latent.weight` (default 1, a purely shared outcome; at $w_s = 1$ nothing is computed, so earlier results are unchanged). At $w_s = 1$ every view carries the same outcome signal; at $w_s = 0$ each view carries its own, independent part. On the Stelzer template at maximum crosstalk (n = 2000, 60 features per view) each view's features explain $R^2 \approx 0.9$ of η at $w_s = 1$, about 0.6 at $w_s = 0.5$ and about 0.35 at $w_s = 0$ (the three views together: 0.95–0.97 throughout). η^{VS} is stored as `Xbeta_specific` and the q_{im} as `truth$specific_scores`.

Interaction. Pairs (a_t, b_t) of features from *different* views (`n.interactions`, default one per view pair; signal features preferred) give

$$\eta_i^{\mathrm{int}} = \sum_t \tilde{x}_{ia_t}\, \tilde{x}_{ib_t}.$$

With `interaction.weight = 1` the outcome has no main effects.

5.5 How the signal controls interact

The controls act on different parts of the model and can be combined to create benchmarking regimes:

Control	Acts on	Effect
`crosstalk` γ	the features (Section 4.2)	how strongly the views share latent structure
`signal.share` w_m	the coefficients	how much outcome signal each view carries
`signal.align` α	the coefficients	whether signal features follow the shared direction (ℓ_j)
`outcome.model`	the predictor	linear, nonlinear, shared-latent or interaction signal

Typical regimes:

- **single-view signal:** `signal.share = c(1, 0, 0)`;
- **complementary signal:** `signal.share = c(1, 1, 1)`, `signal.align = 0` (each view carries its own, unrelated part of the signal);
- **redundant (aligned) signal:** `signal.align = 1` with high crosstalk (the views carry the same signal along the shared direction);
- **cross-view latent signal:** `outcome.model = "shared_latent"`; the outcome depends on the shared factors, and crosstalk sets how strongly they are visible in each view (with $\gamma = 0$ they are invisible, and very high crosstalk makes the views redundant);
- **shared versus view-specific latent signal:** `shared.latent.weight` moves a shared-latent outcome from redundant views ($w_s = 1$) to complementary views ($w_s = 0$), independently of crosstalk. In checks on the Stelzer template (n = 300, maximum crosstalk, lasso base learners), late fusion (per-view models, stacked) beat early fusion (one model on all features) at $w_s = 1$, where it averages several noisy proxies of the same signal, and lost at $w_s = 0$, where each per-view model sees only part of the signal; continuous and survival outcomes behaved the same way;
- **nonlinear or interacting signal:** `"friedman"`, `"friedman2"`, `"interaction"`.

Using several shared factors (`shared.latent.n.factors` > 1) lets the outcome depend on several cross-view axes (for example an immune and a metabolic axis) rather than one. Requesting more factors than the template has is an error, and the Friedman forms need at least five, so fit the template with `shared.rank = 5` for them.

5.6 Outcome types

Continuous.

$$Y_i = \eta_i + \varepsilon_i, \qquad \varepsilon_i \sim N(0,\ \mathrm{Var}(\eta)/\mathrm{snr}),$$

so `snr` $= \mathrm{Var}(\eta)/\mathrm{Var}(\varepsilon)$.

Binary. A logistic model is a threshold model $Y = \mathbf{1}(\eta + \varepsilon > 0)$ with ε standard logistic, $\mathrm{Var}(\varepsilon) = \pi^2/3$. Rescaling the predictor to variance $\mathrm{snr} \cdot \pi^2/3$ gives `snr` the same meaning as above:

$$\eta_i^* = b_0 + \mathrm{std}(\eta)_i \sqrt{\mathrm{snr}\,\pi^2/3}, \qquad Y_i \sim \mathrm{Bernoulli}(\mathrm{logit}^{-1}(\eta_i^*)),$$

with b_0 solved so that $\frac{1}{n}\sum_i \mathrm{logit}^{-1}(\eta_i^*) =$ `binary.prevalence`.

Survival. For exponential event times, $\log T = -\log h_0 - \eta + \log E$ with $E \sim \mathrm{Exp}(1)$, and $\log E$ has a Gumbel distribution with variance $\pi^2/6$; hence

$$\eta_i^* = \mathrm{std}(\eta)_i \sqrt{\mathrm{snr}\,\pi^2/6}, \qquad T_i = E_i^{1/k}, \quad E_i \sim \mathrm{Exponential}(\text{rate } h_0 e^{\eta_i^*}), \qquad h_0 = \texttt{baseline.hazard}.$$

With $k =$ `baseline.shape` (default 1, exponential) the event times are Weibull with hazard $h_0\, k\, t^{k-1} e^{\eta_i^*}$: still proportional hazards, with a hazard that rises ($k > 1$) or falls ($k < 1$) over follow-up. Since $\log T = (-\log h_0 - \eta^* + \log E)/k$, signal and error variance both scale by $1/k^2$ and `snr` keeps its meaning.

Censoring. The default rule is unchanged: $C_i \sim U(\texttt{cens.lower}, \texttt{cens.upper})$ (defaults 1 and 3), or, with $\texttt{cens.rate} = c$, $C_i = u_i\kappa$ with $u_i \sim U(0,1)$ and κ solved so that $\frac{1}{n}\sum_i \mathbf{1}(u_i\kappa < T_i) = c$ ($c = 0$: no censoring). In general a censoring time is

$$C_i = \kappa\, V_i\, e^{-\delta z_i}, \qquad \tfrac{1}{n}\textstyle\sum_i \mathbf{1}(C_i < T_i) = c,$$

with z the standardized predictor, $\delta = \texttt{cens.dependence}$ (default 0) and V_i from the mechanism `cens.type`:

`cens.type`	V_i	Meaning
`"uniform"` (default)	$U(0,1)$	censoring spread evenly over follow-up
`"exponential"`	Exp(1)	constant loss to follow-up
`"administrative"`	$\min\{1 - a\,U_i,\ D_i\}$, $D_i \sim \text{Exp}(-\log(1-d))$	entry uniform over the first fraction $a = \texttt{cens.accrual}$ (0.5) of a study that ends at κ; dropout before the end with probability $d = \texttt{cens.dropout}$ (0)

User times `cens.times` (one time, one per sample, or a function of n; `Inf` = never censored) replace V_i; without `cens.rate` they are used as given ($\kappa = 1$). The solve is on $\log\kappa$ by root finding, and the achieved proportion matches c to within one subject. With $\delta > 0$, higher-risk subjects are censored earlier: censoring is informative, the independent-censoring assumption of the Cox model, Harrell's C and IPCW time-dependent AUC is violated, and these metrics are biased (in checks the true predictor's C and AUC rose). Then $\texttt{time}_i = \min(T_i, C_i)$ and $\texttt{status}_i = \mathbf{1}(T_i \le C_i)$; with `keep.true.times = TRUE`, T_i and C_i are stored as `event_time` and `cens_time`.

5.7 Truth and split

`beta` holds β_j (set to 0 where the linear predictor does not enter the outcome: `"friedman"`, `"shared_latent"`, or a weight of 1); `friedman_term` and `interaction_term` flag the nonlinear and interaction features; `outcome_feature` flags every feature that drives the outcome. The final predictor and its components are stored as `Xbeta`, `Xbeta_linear`, `Xbeta_shared`, `Xbeta_friedman`, `Xbeta_interaction` and (with `shared.latent.weight` < 1) `Xbeta_specific`. A random round($n \cdot$ `train.fraction`) samples (default 70%) form the training set.

6. Crosstalk calibration and template validation

6.1 Block distance

Simulated features are aligned with the observed ones by their template feature (offspring are not compared). For views $a \le b$, with R^{obs}_{ab} and R^{sim}_{ab} the blocks of feature-feature Pearson correlations (diagonal excluded within a view),

$$d_{ab} = \sqrt{\frac{1}{|\mathcal{F}|}\sum_{(i,j)\in\mathcal{F}} \left(R^{\text{sim}}_{ab,ij} - R^{\text{obs}}_{ab,ij}\right)^2},$$

the root-mean-square correlation difference over the finite entries $\mathcal{F}$.

6.2 Calibration (`calibrate_crosstalk()`)

For each grid value γ (default $0, 0.25, \ldots, 1$; values above `crosstalk_max()` are dropped) and replicate $b = 1, \ldots,$`nrep` (default 5), an independent null data set is simulated and

$$D_b(\gamma) = \frac{1}{\binom{M}{2}} \sum_{a<b'} d_{ab'}, \qquad \bar{D}(\gamma) = \frac{1}{\texttt{nrep}} \sum_b D_b(\gamma), \qquad \mathrm{SE}(\gamma) = \frac{\mathrm{sd}_b\, D_b(\gamma)}{\sqrt{\texttt{nrep}}}.$$

The selected value is $\hat{\gamma} = \arg\min_\gamma \bar{D}(\gamma)$. Values with $\bar{D}(\gamma) \le \bar{D}(\hat{\gamma}) + \mathrm{SE}(\hat{\gamma})$ are `near_optimal`, and `selection_frequency` gives how often each value is best in individual replicates. Only cross-view blocks enter the criterion because crosstalk changes only cross-view dependence. Calibration needs at least two views and two features per view; the MAE wrappers run it with seed + 1. With small templates the curve is often shallow; then report results over a range of crosstalk values.

6.3 Validation (`validate_template()`)

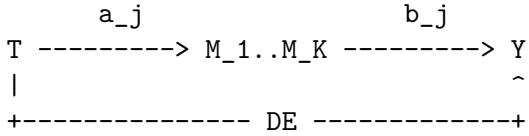
After calibrating, one null data set is simulated at $\hat{\gamma}$ and the `max.features.per.view` (default 100) most variable observed features are compared with the same simulated features:

- **reconstruction error**: the block distances d_{ab};
- **correlation-pattern preservation**: per block, the Pearson and Spearman correlation between the observed and simulated correlations, and their MAE and RMSE;
- **marginals**: per feature the Kolmogorov-Smirnov distance $\sup_x |\hat{F}_{\mathrm{obs}}(x) - \hat{F}_{\mathrm{sim}}(x)|$, and the quantile RMSE over the probabilities $0.05, 0.10, \ldots, 0.95$, raw and divided by the observed SD; summarized per view.

For two independent samples of 53 from the same distribution the KS distance averages about 0.16, so values of that size reflect sampling noise only.

7. Mediation simulation (`gen_mediation()`)

`gen_mediation()` simulates an exposure T, mediators spread over one or more views, and an outcome Y on top of realistic null data, with the true natural direct, indirect and total effects known exactly.



T is randomized; the template's own dependence among features is the only other structure, and Y depends only on T and the mediators.

7.1 Exposure and mediators

$T \sim \mathrm{Bernoulli}(\pi_T)$ (`exposure.prevalence`, default 0.5) or $T \sim N(0,1)$; its standardized version is $T^* = (T - \mu_T)/\sigma_T$ with $(\mu_T, \sigma_T) = (\pi_T, \sqrt{\pi_T(1-\pi_T)})$ or $(0,1)$. Effects are for the contrast $T = 1$ versus $T = 0$.

`n.mediators` mediators per view are drawn among the simulated features whose parent varies enough: the share of template samples not at the feature's most common value must be at least

`min.variability` (default 0.2), so near-constant features (for example taxa that are almost always zero) cannot be mediators.

7.2 Exposure to mediators (a-paths)

The exposure acts on the mediator's latent normal score Z_j before it is mapped to data:

$$W_j = a_j T^* + \sqrt{1 - a_j^2}\, Z_j, \qquad M_j = Q_j(\Phi(W_j^\star)),$$

where $W_j^\star = W_j$ for a continuous exposure (then $W_j \sim N(0,1)$ exactly, and a_j is the latent correlation between exposure and mediator). For a binary exposure W_j is a two-component normal mixture with CDF

$$G_j(w) = (1 - \pi_T)\, \Phi\left(\frac{w - a_j t_0^*}{\sqrt{1 - a_j^2}}\right) + \pi_T\, \Phi\left(\frac{w - a_j t_1^*}{\sqrt{1 - a_j^2}}\right),$$

and $W_j^\star = \Phi^{-1}(G_j(W_j))$.

Proposition. $W_j^\star \sim N(0,1)$ exactly. *Proof.* Given $T^* = t^*$, $W_j \sim N(a_j t^*, 1 - a_j^2)$, so the marginal CDF of W_j is G_j, which is continuous and increasing; hence $G_j(W_j)$ is uniform and Φ^{-1} of a uniform is standard normal. □

The map $w \mapsto \Phi^{-1}(G_j(w))$ is fixed (it depends only on a_j, π_T and the two exposure values) and increasing, so it is the same in every counterfactual world and preserves the sign of the exposure effect. Consequently **every mediator keeps its template marginal** (zeros, bounds, skewness); the exposure changes where each sample falls within it. Adding the effect on the data scale instead ($M \leftarrow M + aT$) would create negative abundances and break point masses.

7.3 Outcome (b-paths)

Mediators enter standardized by their *template* mean and SD on the fitted scale, $\tilde{M}_j = (M_j - \mu_j)/\sigma_j$ (fixed constants, so the outcome model is identical in every counterfactual world):

$$\text{continuous:} \quad Y = \mathrm{DE} \cdot T + \sum_j b_j \tilde{M}_j + \sum_k g_k\, \tilde{M}_{k_1} \tilde{M}_{k_2} + \sigma\varepsilon, \quad \varepsilon \sim N(0,1),$$

$$\text{binary:} \quad \operatorname{logit} P(Y = 1) = b_0 + \mathrm{DE} \cdot T + \sum_j b_j \tilde{M}_j + \sum_k g_k\, \tilde{M}_{k_1} \tilde{M}_{k_2}.$$

The optional interactions (`n.interactions`, default 0) use random pairs of mediators, possibly from different views.

7.4 Natural effects

With potential outcomes $Y(t, m)$ and potential mediators $M(t)$, and because $\mathrm{E}[Y(t,m)]$ has no exposure-mediator interaction,

$$\mathrm{NIE} = \mathrm{E}[Y(1, M(1))] - \mathrm{E}[Y(1, M(0))] = \sum_j b_j \Delta_j + \sum_k g_k \Delta_k,$$
$$\mathrm{NDE} = \mathrm{E}[Y(1, M(0))] - \mathrm{E}[Y(0, M(0))] = \mathrm{DE}, \qquad \mathrm{TE} = \mathrm{NIE} + \mathrm{NDE},$$

with

$$\Delta_j = \mathrm{E}[\tilde{M}_j(1)] - \mathrm{E}[\tilde{M}_j(0)], \qquad \Delta_k = \mathrm{E}[\tilde{M}_{k_1}\tilde{M}_{k_2}(1)] - \mathrm{E}[\tilde{M}_{k_1}\tilde{M}_{k_2}(0)].$$

7.5 Path sizes and exact calibration

Initial sizes. All paths start at

$$s = \sqrt{\mathrm{TIE} \cdot \sigma_T / K}, \qquad a_j = \pm s, \quad b_j = \pm s,$$

with signs from `dir.a` and `dir.b`. The heuristic: a unit contrast moves T^* by $1/\sigma_T$, hence each standardized mediator by about s/σ_T, so NIE $\approx K \cdot s \cdot s/\sigma_T = \mathrm{TIE}$. If $s \geq 0.95$ the request is infeasible (the exposure would explain almost all of a mediator's latent variance) and an error asks for more mediators or a smaller `TIE`. Interaction coefficients start at $\pm\,\mathrm{mean}|b|$.

Calibration. The heuristic is approximate (quantile maps are nonlinear), so the Δ's are computed by Monte Carlo (Section 7.6) and, because the NIE is linear in (b, g) for fixed a-paths, all b and g are multiplied by one factor:

$$\lambda = \frac{\mathrm{TIE} \cdot \mathrm{dir.a} \cdot \mathrm{dir.b}}{\sum_j b_j \Delta_j + \sum_k g_k \Delta_k}, \qquad b \leftarrow \lambda b,\ g \leftarrow \lambda g,$$

so the NIE equals $\pm$TIE exactly. With `"random"` directions the pathway effects can cancel, so the summed absolute effects $\sum_j |b_j \Delta_j| + \sum_k |g_k \Delta_k|$ are set to `TIE` instead.

Null scenarios. `"null_a"`: $a = 0$ with the b-paths of the spiked scenario; `"null_b"`: spiked a, $b = g = 0$; `"null_ab"`: both 0. The NIE is 0 in all three, and power and type I error runs have matched path sizes.

7.6 Paired counterfactual Monte Carlo

The features and mediators of a replicate are fixed. The `n.mc` draws (default 5000) are split into chunks of 1000, each with its own seed. A *counterfactual world* is a full null simulation of a chunk, with that chunk's seed, in which the exposure is set to a given value; all worlds of a chunk use the same seed, so every latent score and noise term is identical across worlds and only the exposure differs (common random numbers). Means of the standardized features are accumulated over chunks. This gives every Δ_j with very low variance and also the true exposure effect on every feature (`exposure_effect`; for non-mediators it is non-zero only through the renormalization of a closed composition).

Continuous exposure. Mediators respond nonlinearly to a continuous exposure, so a population draw $T \sim N(0, 1)$ gives each mediator's linear slope $\mathrm{a_slope}_j = \mathrm{Cov}(T, \tilde{M}_j)/\mathrm{Var}(T)$ and

$$\mathrm{NIE}_{\mathrm{linear}} = \sum_j b_j\, \mathrm{a_slope}_j,$$

the quantity product-of-coefficients estimators target.

7.7 Binary outcome

`TIE` and `DE` are on the conditional log-odds scale. The intercept b_0 is solved so that the population prevalence equals `outcome.prevalence`:

$$\pi_T \,\mathrm{E}[\mathrm{logit}^{-1}(b_0 + \eta(1, M(1)))] + (1 - \pi_T)\,\mathrm{E}[\mathrm{logit}^{-1}(b_0 + \eta(0, M(0)))] = \texttt{outcome.prevalence}$$

(for a continuous exposure, the population draw is used). Odds ratios are non-collapsible, so the natural effects on the probability scale are reported from the counterfactual risks $p_{tt'} = \mathrm{E}[P(Y = 1 \mid T = t, M(t'))]$:

$$\mathrm{RD}_{\mathrm{NIE}} = p_{11} - p_{10}, \quad \mathrm{RD}_{\mathrm{NDE}} = p_{10} - p_{00}, \quad \mathrm{OR}_{\mathrm{NIE}} = \frac{\mathrm{odds}(p_{11})}{\mathrm{odds}(p_{10})}, \quad \mathrm{OR}_{\mathrm{NDE}} = \frac{\mathrm{odds}(p_{10})}{\mathrm{odds}(p_{00})},$$

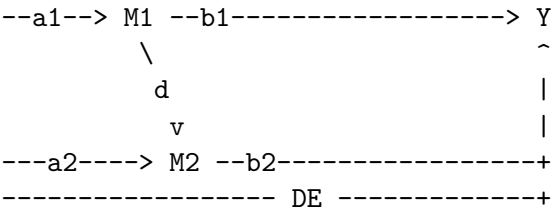
with the total effects $p_{11} - p_{00}$ and $\mathrm{odds}(p_{11})/\mathrm{odds}(p_{00})$, and the log odds ratios.

7.8 Observed data and truth

The observed replicate draws T for `n` samples, applies the a-paths at the observed T^*, and generates Y from the model above (always on the fitted scale). Per feature: `mediator`, `a_path` (a_j), `a_effect` (Δ_j), `a_slope`, `b_path`, `indirect` ($b_j\Delta_j$), `exposure_effect`, `interaction_term`. In `truth$mediation`: `NIE`, `NDE`, `TE`, `TIE_target`, `NIE_linear`, `NIE_by_view`, `NIE_interaction`, and for a binary outcome the risks, risk differences, odds ratios and log odds ratios.

8. Sequential mediation

With `stages`, mediators are grouped into ordered stages of views (for example `list("Microbiome", "Metabolome")` for exposure → microbes → metabolites → outcome), and each mediator of a later stage is linked to mediators of the previous stage.



8.1 Latent model

For a mediator j of stage s with parents k in stage $s-1$:

$$s = 1: \quad W_j = a_j T^* + \sqrt{1 - a_j^2}\, Z_j,$$
$$s > 1: \quad W_j = a_j T^* + \sum_k d_{jk} W_k + c_j Z_j.$$

Parents enter through their own latent scores *after* their exposure effect, so exposure that reached a parent is passed on. Because each W is linear in T^* and the template scores $Z = (Z_1, \dots, Z_K)$,

$$W_j = \mu_j T^* + L_j Z, \qquad \mu_j = a_j + \sum_k d_{jk}\mu_k, \qquad L_j = \sum_k d_{jk} L_k + c_j e_j,$$

with $\mu_j = a_j$ and $L_j = \sqrt{1 - a_j^2}\, e_j$ in stage 1.

Template correlation. The Z's of different mediators are correlated through the template (within a view by the factor model, across views by the shared space). Their covariance is computed exactly: for mediator i in view m and l in view m',

$$\Sigma_{il} = \Lambda_i \, C_{mm'} \, \Lambda_l^\top, \qquad C_{mm} = I, \qquad C_{mm'} = \sqrt{\gamma_m \gamma_{m'}}\, B_m B_{m'}^\top.$$

Unit variance. With $L_0 = \sum_k d_{jk} L_k$, $v_0 = L_0 \Sigma L_0^\top$ and $\kappa = L_0 \Sigma e_j$, the condition $\mathrm{Var}(W_j) = \mu_j^2 + v_0 + 2c_j\kappa + c_j^2 = 1$ gives the positive root

$$c_j = -\kappa + \sqrt{\kappa^2 + 1 - \mu_j^2 - v_0}$$

(an error if it does not exist or is below 10^{-3}). Then, given T, $W_j \sim N(\mu_j T^*, 1 - \mu_j^2)$, and the binary-exposure correction of Section 7.2 is applied with μ_j in place of a_j. Every mediator at every stage keeps its template marginal.

8.2 Links

The links from stage $s-1$ to stage s form a sparse signed matrix (rows: children, columns: parents), many-to-many. `stage.links$type`:

- `"random"`: each pair linked with probability `density` (0.3); every child gets at least one parent and every parent at least one child;
- `"hub"`: $\max(1, \mathrm{round}(\mathrm{density} \cdot n_{\mathrm{parents}}))$ parents are hubs, every child has exactly one hub parent;
- `"convergent"`: every child has $\max(2, \mathrm{round}(\mathrm{density} \cdot n_{\mathrm{parents}}))$ parents;
- `"explicit"`: the user supplies the matrices.

Each child's pattern row w_j is scaled so that the parents' term has variance `strength` (default 0.3):

$$d_j = w_j \sqrt{\frac{\texttt{strength}}{w_j^\top R\, w_j}}, \qquad R = \mu_{\mathcal{P}} \mu_{\mathcal{P}}^\top + L_{\mathcal{P}} \Sigma L_{\mathcal{P}}^\top,$$

with R the covariance of the parents' latent scores, so the strength does not depend on how many parents a child has. The links are fixed in the spiked scenario and reused unchanged in the null scenarios. `stage.direct` multiplies the initial a-paths of each stage (0: the stage is reached only through the previous one).

8.3 Natural path-specific effects

With S stages there are $S(S+1)/2$ indirect paths (r, s), $r \le s$: the exposure enters at stage r and reaches Y through the stage-s mediators. For each observed stage s, the chain up to s is built from its own copy of the exposure, in which stage r receives the exposure value $t(r, s)$; all copies share the same template noise (cross-world counterfactuals with common random numbers). The paths are switched from 0 to 1 one at a time, later starting paths first, and each path's effect is the change in the expected outcome. For two stages:

$$\begin{aligned} T \to M_2 \to Y: &\quad \mathrm{E}\,Y(1, M_1(0), M_2(1, M_1(0))) - \mathrm{E}\,Y(1, M_1(0), M_2(0, M_1(0))), \\ T \to M_1 \to M_2 \to Y: &\quad \mathrm{E}\,Y(1, M_1(0), M_2(1, M_1(1))) - \mathrm{E}\,Y(1, M_1(0), M_2(1, M_1(0))), \\ T \to M_1 \to Y: &\quad \mathrm{E}\,Y(1, M_1(1), M_2(1, M_1(1))) - \mathrm{E}\,Y(1, M_1(0), M_2(1, M_1(1))). \end{aligned}$$

These telescope, so they add up exactly to the NIE.

Calibration. Given the a-paths and links, each path effect is linear in the b-paths, and paths ending at stage s scale with the stage-s b-paths. By default one factor scales all b-paths (as in Section 7.5). With `stage.share` $= (\omega_1, \dots, \omega_S)$, normalized,

$$b_j \leftarrow b_j\, \lambda_{s(j)}, \qquad \lambda_s = \frac{\omega_s \cdot \text{target}}{\text{raw}_s},$$

where raw_s is the sum of the path effects ending at stage s before scaling; the mediators of stage s then carry exactly the share ω_s (exact without interactions). Net calibration to $\pm$TIE needs positive links; with negative or random links the summed absolute pathway effects are calibrated.

Truth. `path_effects` (one row per path), `NIE_by_stage`, `links` (parent, child, latent coefficient d_{jk} and approx_chain_effect $= (\mu_{\text{parent}}/\sigma_T)\, d_{jk}\, b_{\text{child}}$, a latent-scale linear approximation), per feature `stage`, `n_parents`, `n_children`, and for a binary outcome the path-specific risk differences.

8.4 Scenarios

Setting	Scenario
defaults	all three paths active
`stage.direct = c(1, 0)`, `stage.share = c(0, 1)`	pure chain $T \to M_1 \to M_2 \to Y$
`stage.links = list(strength = 0)`	unlinked stages, still correlated through crosstalk (a null for the link)
`effect.type = "null_a"`, `"null_b"`, `"null_ab"`	as in parallel mediation; links unchanged

8.5 Estimation on the observed scale

The truth is exact, but what an estimator recovers from observed data is a different question:

- **Crosstalk is a mediator-mediator confounder.** The template scores of the stage-1 and stage-2 mediators are correlated through the shared variates, a common cause of both that is not on any path from T. The path-specific truth holds it fixed, but a regression of M_2 on T and M_1 is confounded by it. On the Stelzer template (n = 30,000, TIE = 1) the chain

$T \to M_1 \to M_2 \to Y$ was 0.23 (truth), estimated as 0.14 with crosstalk 1 and 0.19 with crosstalk 0; the difference moved to the direct $T \to M_2 \to Y$ path.

- **Nonlinear marginals.** For skewed or sparse parents (microbiome relative abundances) the observed value is a nonlinear function of the latent score, so a linear model captures only part of the chain; on the Franzosa microbiome about half of the chain was attributed to the direct path. Transforming such features (ranks, log ratios) helps.

9. Multi-study simulation (`gen_multistudy()`)

`gen_multistudy()` simulates S studies that measure the same simulated features, from one template or from one template per study (one real cohort each, fitted together with `fit_study_templates()`). With one study and no heterogeneity it gives the same features, coefficients and outcome as `gen_simmba()` with the same seed; both run on one outcome engine.

Heterogeneity	Model	Argument
Case mix	shift of the study's latent factor means; enters the outcome	`shift.sd`
Batch	$z \to a + bz$ on each feature's latent score; does not enter the outcome	`batch.loc.sd`, `batch.scale.sd`
Signal features	fraction of each study's signal features from the common set	`support.overlap`
Effect sizes	correlation of the common features' effects between studies	`effect.cor`
Baseline and noise	study intercept and log-SNR spread	`intercept.sd`, `snr.sd`

9.1 Features of every study

Study 1 chooses the simulated features; the other studies reuse them. With one template per study, the choice is carried to each template by feature name (the same parents, and for offspring the same parent and partner features with that template's loadings), so every study keeps its own cohort's marginals and correlation structure. `fit_study_templates()` keeps the views and features that every cohort has, drops features that are missing or constant in any cohort, and refits until all templates have identical features.

Case mix. The factor scores of view m are $F_m = \sqrt{\gamma_m}\, S B_m^\top + E_m R_m$ with $R_m = (I - \gamma_m B_m B_m^\top)^{1/2}$. Study s shifts the shared variates by δ_s^{sh} and the view innovations by $\delta_{s,m}^E$, all $N(0, \sigma^2_{\mathrm{shift}})$, so the scores move by

$$d_{s,m} = \sqrt{\gamma_m}\, B_m\, \delta_s^{\mathrm{sh}} + R_m\, \delta_{s,m}^E, \qquad \mathrm{Cov}(d_{s,m}) = \sigma^2_{\mathrm{shift}} I,$$

and the latent feature scores by $\Lambda_m d_{s,m}$. The shift is biology: it is part of the latent truth and moves the outcome.

Batch. For study s and feature j,

$$z_{ij} \to a_{sj} + b_{sj}\, z_{ij}, \qquad a_{sj} \sim N(0, \sigma_a^2), \quad \log b_{sj} \sim N(0, \sigma_b^2),$$

before the template marginal, so values stay in the observed support (zeros included; in two-part views presence probabilities change too). The engine maps both the batch-free and the batch-affected

latent scores; the outcome is built from the batch-free features, so batch effects are pure technical noise.

9.2 Study coefficients

From the common coefficients β (Section 5.2), study s keeps a random round(ωk_m) of the k_m common signal features of view m ($\omega =$ `support.overlap`) and draws the rest from the other features of the view, following `signal.align`. Two studies then share about $\omega^2 k_m$ common features, a Jaccard overlap of about $\omega^2/(2-\omega^2)$. For a kept feature,

$$\beta_{sj} = \mu + \sqrt{\rho}\,(\beta_j - \mu) + \sqrt{1-\rho}\,(\beta^*_{sj} - \mu),$$

with $\rho =$ `effect.cor`, β^*_{sj} a fresh draw from the effect-size distribution and $\mu = (1-2q)\,\mathbb{E}|L|$ its mean ($q =$ `signal.neg.prob`, $L = \log_2 f$ normal, so $\mathbb{E}|L|$ is a folded-normal mean). Then $\mathbb{E}\beta_{sj} = \mu$, $\operatorname{Var}\beta_{sj} = \operatorname{Var}\beta_j$ and $\operatorname{Cor}(\beta_{sj}, \beta_{tj}) = \rho$.

9.3 Predictor and outcome

Features are standardized over all studies (within-study standardization would erase the study differences). Study s's linear predictor uses β_s; Friedman features, interaction pairs and shared-latent factors are common to all studies; `signal.share` rescales each view by one factor for all studies. With V the pooled within-study variance of η, $\alpha_s \sim N(0, \sigma_\alpha^2)$ and $\mathrm{SNR}_s = \mathrm{snr} \cdot e^{\epsilon_s}$, $\epsilon_s \sim N(0, \sigma^2_{\mathrm{snr}})$:

- continuous: $Y = \eta + \alpha_s\sqrt{V} + \varepsilon$, $\varepsilon \sim N(0, V/\mathrm{SNR}_s)$;
- binary: $\operatorname{logit} P(Y=1) = c + \alpha_s + \sqrt{\mathrm{SNR}_s\,\pi^2/3}\,(\eta - \bar\eta)/\sqrt{V}$, with c solved so the pooled prevalence is `binary.prevalence`;
- survival: hazard $h_0\,k\,t^{k-1}\exp\{\alpha_s + \sqrt{\mathrm{SNR}_s\,\pi^2/6}\,(\eta-\bar\eta)/\sqrt{V}\}$, censoring solved within each study; `cens.rate` may give one target per study (different follow-up), and `cens.times` a list with one entry per study.

Studies in `holdout.studies` go entirely to the test set; the others are split by `train.fraction`.

9.4 Random streams

Heterogeneity is drawn from side streams: a seed derived from the current random state plus a tag, without advancing the main stream. With the same seed, the base data, common coefficients and noise are therefore identical at every heterogeneity level, and batch effects leave the outcome exactly unchanged.

9.5 Validation

Sweeps with 30 replicates per value (Stelzer and Franzosa as single templates; PRISM, Validation and Lloyd-Price baseline from HuMMANet as per-study templates): the realized coefficient correlation follows `effect.cor` (0.47 to 0.51 at 0.5); cross-study prediction R^2 rises from about 0.06 to 0.31–0.54 while within-study prediction stays flat; the share of feature variance explained by study rises monotonically with `shift.sd` and `batch.loc.sd`; within-study correlations change by less than replicate-to-replicate noise; the outcome is unchanged by batch. On sparse microbiome templates, keep `shift.sd` and the batch SDs at about 0.5 or below: larger values make rare taxa constant within a study.

10. MAE wrappers and reproducibility

`simulate_mimesys()`, `simulate_mimesys_null()`, `simulate_multistudy()` and `simulate_mediation()` fit the template if needed, calibrate crosstalk by default (`crosstalk = "calibrate"`; skipped for a single view, no shared factors, or fewer than two features per view), call the simulators, and return one `MultiAssayExperiment` per replicate: assay `simulated`, feature truth in `rowData()`, outcome, exposure and split in `colData()`, settings, provenance and latent truth in `metadata()`.

Every function that uses random numbers has `seed = 1234` by default and restores the caller's random-number state on exit; `seed = NULL` uses the current stream. Calibration inside the wrappers uses seed + 1, so its replicates never share random numbers with the simulation, and internal loops call unseeded engines so replicates are independent.

11. Worked example: the Stelzer template

`fit_template(stelzer_mae, view.transform = c(Metabolomics = "log1p", Proteomics = "log1p"))` (53 subjects, 40 features in each of CYTOF, Metabolomics and Proteomics):

- within-view ranks $k = (6, 7, 7)$, explaining 0.72, 0.73 and 0.70 of the normal-score variance;
- largest GCCA eigenvalues 2.12, 1.81, 1.75, 1.67, 1.56;
- parallel analysis: step 1 tests $\lambda = 2.121$ against a null 95th percentile of 2.083 ($p = 0.020$, kept); after deflation, step 2 tests $\lambda = 1.808$ against 2.007 ($p = 0.667$, stop), so $r = 1$. An index-matched rule would have kept several of the slowly decreasing eigenvalues;
- `crosstalk_max()` $= (1.42, 1.46, 1.37)$;
- calibration over $\gamma \in \{0, 0.125, \ldots, 1\}$ with 10 replicates gives mean cross-view distances between 0.192 and 0.206 (minimum at $\gamma = 0.625$): a shallow curve, because with 53 samples the sampling noise of a correlation ($\approx 1/\sqrt{53} = 0.14$) dominates the weak cross-view signal;
- validation: mean KS distances 0.10 to 0.15 per view, at the level expected for two samples of 53 from the same distribution.

On the Franzosa template (155 samples, Microbiome and Metabolome), the parallel-analysis steps have $p = 0.005, 0.005, 0.010, 0.388$, so $r = 3$, and the Microbiome view is a proportion view with largest total 0.917.

For a controlled nonlinear shared-latent setting:

```
template <- fit_template(views, rank = 10, shared.rank = 5)
sim <- gen_simmba(
  n = 200, template = template, p = 100, crosstalk = 0.75,
  outcome.model = "shared_latent", shared.latent.function = "friedman2",
  shared.latent.n.factors = 5, snr = 0.5
)
```

Five shared factors are fitted and all five drive the outcome (linear plus nonlinear parts), and crosstalk sets how visible they are in each view.

12. Default settings

Function	Main defaults
`fit_template()`	`rank = NULL`, `var.explained = 0.7`, `max.rank` $= \lfloor n_0/3 \rfloor$, `shared.rank = NULL`, `nperm = 200`, `shared.rank.runs = 1`
`sim_from_template()`	`crosstalk = 1`, `feature.select = "random"`, `feature.jitter = 0.2`, `back.transform = FALSE`
`gen_simmba()`	`signal.prob = 0.1`, `signal.neg.prob = 0.5`, `effect.meanlog = 1`, `effect.sdlog = 0.4`, `outcome.model = "linear"`, `outcome.type = "continuous"`, `snr = 1`, `binary.prevalence = 0.5`, `nonlinear.weight = interaction.weight = 0.5`, `shared.latent.weight = 1`, `baseline.hazard = baseline.shape = 1`, `cens.rate = NULL`, `cens.type = "uniform"`, `cens.dependence = 0`, `cens.accrual = 0.5`, `cens.dropout = 0`, `keep.true.times = FALSE`, `train.fraction = 0.7`, `nrep = 1`
`calibrate_crosstalk()`	`crosstalk.grid = seq(0, 1, 0.25)`, `nrep = 5`, `n` $= n_0$
`gen_multistudy()`	`effect.cor = 1`, `support.overlap = 1`, `intercept.sd = snr.sd = shift.sd = batch.loc.sd = batch.scale.sd = 0`, `holdout.studies = NULL`; other defaults as `gen_simmba()`
`gen_mediation()`	`n.mediators = 5`, `TIE = 1`, `DE = 0`, `exposure.type = "binary"`, `exposure.prevalence = 0.5`, `n.mc = 5000`, `min.variability = 0.2`, `stage.links`: density 0.3, strength 0.3

13. Assumptions and limitations

- Dependence is a Gaussian copula: linear on the latent scale; nonlinear and tail dependence are not reproduced.
- Within-view dependence is low-rank, and cross-view dependence is limited to the r estimated shared directions; crosstalk scales them but cannot create new ones.
- Simulated values stay within the template's observed range.
- Samples are independent; repeated measures must be reduced to one sample per subject.
- Small templates give noisy shared structure and shallow calibration curves.
- Planted outcomes are controlled experiments, not imitations of the real outcome mechanism.
- Survival outcomes have proportional hazards (exponential or Weibull baseline); non-proportional hazards are not simulated.
- Mediation does not include confounders, survival outcomes or count data with library sizes, and simulates one study.
- In multi-study simulation, case-mix and batch shifts act on the latent scale, so on sparse

features they also change spread and zero fractions; with per-study templates of different ranks the latent truth is not returned.

References

- Daniel RM, De Stavola BL, Cousens SN, Vansteelandt S (2015). Causal mediation analysis with multiple mediators. *Biometrics* 71:1-14.
- Friedman JH (1991). Multivariate adaptive regression splines. *Annals of Statistics* 19:1-67.
- Horn JL (1965). A rationale and test for the number of factors in factor analysis. *Psychometrika* 30:179-185.
- Kettenring JR (1971). Canonical analysis of several sets of variables. *Biometrika* 58:433-451.
- Zappia L, Phipson B, Oshlack A (2017). Splatter: simulation of single-cell RNA sequencing data. *Genome Biology* 18:174.